



UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA



DISIM
Dipartimento di Ingegneria
e Scienze dell'Informazione
e Matematica

Dipartimento di Ingegneria e Scienze dell'Informazione
e Matematica (DISIM)

ADVANCED AND SOFTWARE DEFINED NETWORKS

Docente: *Prof. Centofanti Carlo*

Studente: **Diego Banfi**

A.A. 2025–2026

Contents

1	Introduzione e Modifica della Topologia	2
2	TASK1: Creazione dell'Anello SDN	2
2.1	Modifiche a <code>lab.conf</code> e ai File <code>.startup</code> per l'Integrazione di s3	4
2.1.1	Modifiche a <code>lab.conf</code>	4
2.1.2	Modifiche a <code>s1.startup</code>	4
2.1.3	Modifiche a <code>s2.startup</code>	4
2.1.4	Nuovo File <code>s3.startup</code>	5
2.1.5	Nota Bene: Porte Kathará, Linux e OpenFlow	6
3	TASK 2: Implementazione del Firewall Stateless Selettivo a Livello 3	7
3.0.1	Filosofia del Filtraggio a Livello 3 in Reti SDN	7
3.0.2	Implementazione della Policy di Firewall nel Controller	8
4	TASK 3: Policy Routing con Rete ad Anello	10
4.1	Architettura logica del controller	11
4.2	Fase 1: Identificazione di Host e Switch	12
4.3	Fase 2: Descrizione della Policy di Routing	12
4.4	Fase 3: Installazione Proattiva delle Flow Rule	13
4.5	Fase 4: Necessità della Gestione ARP	14
4.6	Esempio di funzionamento di ARP sull'anello	15
4.6.1	Il problema centrale: sovrascrittura delle porte e corruzione della memoria di Ryu	16
4.7	Fase 5: Identificazione delle Porte Inter-Switch	17
4.8	Fase 6: Apprendimento ARP e Proxy ARP	18
4.9	Fase 7: ARP REQUEST CON FLOODING CONTROLLATO ANTI-LOOP	19
4.10	Fase 8: ARP Reply e MAC Learning generico	20
5	Validazione sperimentale e risultati	21
5.1	Ambiente di test e metodologia	21
5.2	Avvio del controller e connessione degli switch	21
5.3	Verifica dello stato iniziale delle Flow Table	22
5.4	Verifica iniziale delle Flow Table	22
5.5	Test di connettività e verifica del firewall	23
5.6	Verifica del policy routing asimmetrico	26
5.7	Test dei servizi TCP: curl e iperf3	26
5.8	Verifica della gestione ARP e PROXY ARP	27

1 Introduzione e Modifica della Topologia

L'obiettivo di questo progetto consiste nell'estensione e nell'ottimizzazione di una rete Software-Defined Networking (SDN) preesistente, originariamente basata sulle specifiche del laboratorio di riferimento. L'infrastruttura di partenza, che implementava un tradizionale comportamento di *Layer 2 Learning Switch* controllato centralmente tramite il framework Ryu, è stata modificata sia a livello topologico sia a livello logico.

Le attività principali si articolano in tre macro-task:

1. **Estensione della topologia:** Inserimento di un terzo nodo di commutazione SDN (*s3*) per la chiusura di un anello ridondante tra i nodi Core.
2. **Sicurezza (Firewalling Hardware):** Implementazione di una politica proattiva di filtraggio del traffico per l'isolamento di un host non autorizzato direttamente sul piano dati.
3. **Traffic Engineering (Policy Routing):** Deviazione chirurgica di un flusso dati unicast specifico su un percorso alternativo sfruttando il nuovo link, eludendo il percorso di default.

2 TASK1: Creazione dell'Anello SDN

Nella topologia originaria, gli switch Open vSwitch (OVS) gestivano l'inoltro del traffico basandosi su un collegamento lineare. Per introdurre tolleranza ai guasti e capacità di bilanciamento del carico, è stato introdotto lo switch OpenFlow *s3*.

Il nuovo nodo *s3* è stato interconnesso direttamente sia con lo switch *s1* (Core) sia con lo switch *s2*. Questa interconnessione introduce una struttura ad anello (loop fisico) all'interno del dominio SDN. In una rete locale tradizionale (composta da switch legacy), una simile topologia provocherebbe istantaneamente una tempesta di broadcast (*Broadcast Storm*), saturando i link e causando il crash dell'infrastruttura in assenza del protocollo Spanning Tree (STP). Nel paradigma SDN, la presenza dell'anello viene gestita in modo deterministico via software grazie al controllo puntuale delle tabelle di flusso.

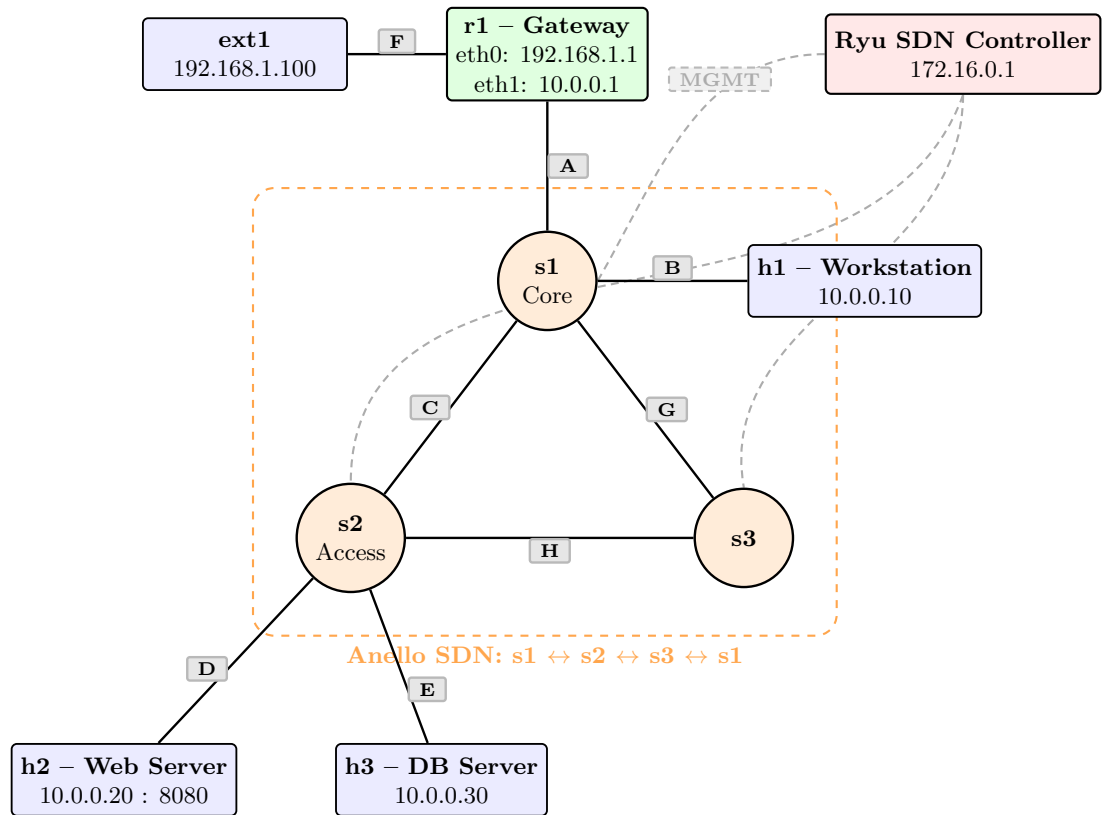


Figure 1: Topologia SDN estesa con anello tra gli switch s1, s2 e s3. Le linee continue rappresentano il data plane e le linee tratteggiate il piano di management verso il controller Ryu. I domini G e H sono i nuovi collision domain introdotti per creare l'anello.

Le uniche interfacce nuove rispetto alla topologia di partenza sono quelle necessarie a collegare s3 al resto della rete:

- Dominio G: s1:eth4 ↔ s3:eth0
- Dominio H: s2:eth4 ↔ s3:eth1
- Management: controller ↔ s3:eth2

2.1 Modifiche a lab.conf e ai File .startup per l'Integrazione di s3

Per integrare lo switch **s3** e chiudere l'anello logico della topologia SDN, sono state introdotte modifiche puntuali sia nel file di configurazione globale **lab.conf** sia nei file **.startup** degli switch **s1** e **s2**. Di seguito sono riportate esclusivamente le righe **aggiunte** rispetto alla topologia lineare di partenza, priva del terzo switch.

2.1.1 Modifiche a lab.conf

Rispetto alla configurazione originale, in cui **s1** e **s2** erano collegati esclusivamente tramite il dominio di collisione **C**, sono state aggiunte le seguenti righe:

Listing 1: Righe aggiunte a lab.conf per l'inserimento di s3

```
# --- Core SDN Switch (s1) ---
s1[4]="G"           # NUOVO: interfaccia eth4, verso s3

# --- Access SDN Switch (s2) ---
s2[4]="H"           # NUOVO: interfaccia eth4, verso s3

# --- New SDN Switch (s3) ---
s3[image]="kathara/sdn"
s3[0]="G"           # Verso s1 (eth0)
s3[1]="H"           # Verso s2 (eth1)
s3[2]="MGMT"        # Interfaccia di Management (eth2)
```

Le righe **s1[4]="G"** e **s2[4]="H"** assegnano una nuova interfaccia Ethernet (**eth4**) rispettivamente a **s1** e **s2**, collegandole ai due nuovi domini di collisione **G** e **H**. Il blocco **s3[...]** definisce interamente il nuovo nodo, con due interfacce dati (**eth0**, **eth1**) verso l'anello e una interfaccia di management (**eth2**).

2.1.2 Modifiche a s1.startup

Nel file di avvio di **s1**, è stata aggiunta l'interfaccia **eth4** al bridge Open vSwitch, corrispondente alla nuova Porta OpenFlow verso **s3**:

Listing 2: Riga aggiunta a s1.startup

```
ovs-vsctl add-port s1 eth4    # NUOVO: OF Porta 4, verso s3
```

2.1.3 Modifiche a s2.startup

Analogamente, in **s2.startup** è stata aggiunta l'interfaccia **eth4** verso **s3**:

Listing 3: Riga aggiunta a s2.startup

```
ovs-vsctl add-port s2 eth4    # NUOVO: OF Porta 4, verso s3
```

2.1.4 Nuovo File s3.startup

Poiché **s3** è un nodo completamente nuovo, il suo file di avvio è stato scritto da zero. Le operazioni eseguite replicano lo stesso schema degli altri due switch: avvio del servizio Open vSwitch, creazione del bridge, aggiunta delle due porte dati, configurazione dell'interfaccia di management e connessione al controller Ryu:

Listing 4: Contenuto di s3.startup

```
# s3.startup      Nuovo SDN Switch (anello)
# Data ports: eth0 -> s1, eth1 -> s2
# Management: eth2 -> controller (NON aggiunta al bridge OVS
)

/etc/init.d/openvswitch-switch start

ovs-vsctl add-br s3

ovs-vsctl add-port s3 eth0
ovs-vsctl add-port s3 eth1

ip link set eth0 up
ip link set eth1 up

ip addr add 172.16.0.30/24 dev eth2
ip link set eth2 up

ovs-vsctl set bridge s3 other-config:datapath-id
=000000000000000003

ovs-vsctl set-controller s3 tcp:172.16.0.1:6653
```

- **Creazione del bridge e configurazione del piano dati:** il comando `add-br s3` crea il bridge logico che rappresenta lo switch Open vSwitch. Successivamente, i comandi `add-port s3 eth0` e `add-port s3 eth1` associano al bridge le due interfacce dati. L'interfaccia `eth0` è collegata a `s1` tramite il dominio di collisione `G`, mentre `eth1` è collegata a `s2` tramite il dominio `H`. Da questo momento, entrambe le interfacce vengono gestite da Open vSwitch come porte del piano dati.
- **Configurazione del canale out-of-band di management:** l'interfaccia `eth2`, collegata al dominio isolato `MGMT`, riceve l'indirizzo IP statico `172.16.0.30/24`. Questa interfaccia non viene aggiunta al bridge `s3`; pertanto non partecipa al forwarding dei pacchetti del piano dati e viene utilizzata esclusivamente per la comunicazione con il controller Ryu.
- **Connessione al piano di controllo:** il comando `ovs-vsctl set-controller s3 tcp:172.16.0.1:6653` configura il controller Ryu come controller Open-Flow dello switch. La comunicazione avviene tramite l'indirizzo `172.16.0.1`

e la porta TCP 6653. Dopo questa configurazione, **s3** può ricevere dal controller le regole di forwarding necessarie per il routing e per le politiche SDN.

- **Assegnazione del Datapath ID:** il comando `other-config:datapath-id` assegna allo switch **s3** il DPID 0000000000000003. Questo identificativo consente al controller Ryu di distinguere **s3** dagli altri switch della rete e di applicare a ciascun dispositivo le corrette regole OpenFlow.

2.1.5 Nota Bene: Porte Kathará, Linux e OpenFlow

Nella configurazione è necessario distinguere tra indice Kathará, nome dell'interfaccia Linux e porta OpenFlow.

La riga seguente in `lab.conf`:

```
s1 [4] = "G"
```

crea sul nodo **s1** l'interfaccia Linux **eth4** e la collega al dominio di collisione **G**. Tale dominio collega **s1** a **s3**:

```
s1:eth4 ↔ G ↔ s3:eth0
```

Per rendere questa interfaccia utilizzabile dal controller SDN, essa viene aggiunta al bridge Open vSwitch:

```
ovs-vsctl add-port s1 eth4
```

Nel laboratorio, le porte dati di **s1** vengono aggiunte al bridge nell'ordine **eth0**, **eth1**, **eth2** ed **eth4**. Di conseguenza, OVS assegna a **eth4** la Porta OpenFlow 4.

Table 1: Porte dati di s1

Interfaccia	Collegamento	Porta OF
eth0	r1	1
eth1	h1	2
eth2	s2	3
eth4	s3	4

Il numero nel nome **eth4** non determina automaticamente il numero della porta OpenFlow: tale nome deriva da Kathará, mentre la porta OpenFlow viene assegnata da OVS quando l'interfaccia viene aggiunta al bridge.

L'interfaccia **eth3** non compare nella tabella perché viene usata solo per la rete di management e non viene aggiunta al bridge OVS. Per questo non viene utilizzata per l'inoltro dei pacchetti nel data plane.

La corrispondenza può essere verificata con:

```
ovs-vsctl get Interface eth4 ofport
```

Nel caso considerato il risultato è 4. Il controller Ryu usa quindi la Porta OpenFlow 4 di **s1** per inoltrare il traffico verso **s3**.

3 TASK 2: Implementazione del Firewall Stateless Selettivo a Livello 3

L'obiettivo del secondo task consiste nell'implementare una politica di sicurezza sul piano dati all'interno del dominio SDN. A differenza di un filtraggio indiscriminato che isolerebbe completamente un host dall'intera rete, in questo scenario si applica un ****filtraggio selettivo e bidirezionale****: inibire completamente la comunicazione IP tra la workstation h_1 (10.0.0.10) e il database server h_3 (10.0.0.30), garantendo e salvaguardando al contempo la piena connettività del web server h_2 (10.0.0.20) verso entrambi gli endpoint e verso l'esterno.

3.0.1 Filosofia del Filtraggio a Livello 3 in Reti SDN

Il principale vantaggio dell'architettura SDN consiste nella separazione tra il piano di controllo e il piano dati. Il controller Ryu rappresenta il punto centrale in cui vengono definite le politiche della rete, mentre gli switch Open vSwitch si occupano di applicare concretamente tali politiche ai pacchetti.

Nel caso del firewall, il controller stabilisce che il traffico tra h_1 (10.0.0.10) e h_3 (10.0.0.30) deve essere bloccato. Questa decisione viene presa centralmente e successivamente trasformata dal controller in una regola OpenFlow, installata direttamente nella *Flow Table* dello switch interessato.

In questo modo, il controller non deve analizzare ogni pacchetto della rete. Dopo aver installato la regola, lo switch può applicarla autonomamente. I pacchetti successivi vengono quindi bloccati direttamente nel piano dati, senza dover essere inviati nuovamente al controller. Questo riduce il numero di messaggi scambiati sul canale di controllo e diminuisce la latenza di elaborazione dei pacchetti successivi.

Il funzionamento della policy segue tre fasi principali:

1. **Fase reattiva – primo pacchetto:** quando h_1 tenta di inviare il primo pacchetto IP verso h_3 (o nella direzione opposta), lo switch non possiede ancora una regola specifica per quel traffico. Il pacchetto incontra quindi la regola di *table-miss*, con priorità pari a 0, e viene inviato al controller Ryu tramite un messaggio `OFF_PACKET_IN`.
2. **Decisione centralizzata del controller:** l'applicazione Ryu analizza il pacchetto, legge gli indirizzi IPv4 sorgente e destinazione e verifica se il traffico appartiene alla comunicazione vietata $h_1 \leftrightarrow h_3$. Poiché la comunicazione deve essere bloccata, il controller non inoltra il pacchetto e invia allo switch un messaggio `OFF_FLOW_MOD`. Questo messaggio installa una regola di *DROP* con priorità 300.
3. **Fase autonoma – pacchetti successivi:** dopo l'installazione della regola, lo switch riconosce autonomamente i pacchetti successivi appartenenti allo stesso flusso. La regola con priorità 300 viene applicata direttamente nella *Flow Table* e il pacchetto viene scartato senza essere inviato

al controller. In questo modo si riduce la latenza, diminuisce il traffico sul canale di controllo e il controller rimane disponibile per gestire altri eventi della rete.

3.0.2 Implementazione della Policy di Firewall nel Controller

Per implementare il Task 2 è stato modificato esclusivamente il file Python dell'applicazione Ryu. La policy definita blocca il traffico IPv4 in entrambe le direzioni tra h_1 e h_3 . Il controllo viene eseguito nel metodo `_packet_in_handler()`, cioè nella funzione invocata dal framework Ryu quando uno switch invia al controller un pacchetto che non corrisponde a una regola già presente nella propria *Flow Table*.

Listing 5: Blocco di codice relativo al firewall del Task 2

```
# Indirizzi IP utilizzati dalla policy
IP_H1 = "10.0.0.10"
IP_H3 = "10.0.0.30"

# Firewall H1 <-> H3
ip_pkt = pkt.get_protocol(ipv4.ipv4)
if ip_pkt:
    src_ip = ip_pkt.src
    dst_ip = ip_pkt.dst

    if ((src_ip == IP_H1 and dst_ip == IP_H3) or
        (src_ip == IP_H3 and dst_ip == IP_H1)):
        self.logger.warning("FIREWALL DROP: %s<->%s",
                             src_ip, dst_ip)

        match = parser.OFPMatch(
            eth_type=0x0800,
            ipv4_src=src_ip,
            ipv4_dst=dst_ip
        )

        self._add_flow(
            datapath,
            priority=300,
            match=match,
            actions=[]
        )
    return
```

Le costanti IP_H1 e IP_H3 identificano gli endpoint coinvolti dalla policy di sicurezza. In particolare, IP_H1 contiene l'indirizzo della workstation h_1 (10.0.0.10), mentre IP_H3 contiene l'indirizzo del database server h_3 (10.0.0.30). L'indirizzo di h_2 non è utilizzato dal firewall, poiché il traffico verso il web server deve rimanere consentito.

L'istruzione:

```
ip_pkt = pkt.get_protocol(ipv4.ipv4)
```

estrae dal pacchetto ricevuto l'header IPv4. Se il pacchetto non è di tipo IPv4, il valore di `ip_pkt` è nullo e il blocco firewall non viene applicato. Questo consente, ad esempio, di trattare separatamente il traffico ARP necessario alla risoluzione degli indirizzi MAC.

L'istruzione condizionale:

```
if ip_pkt:
```

verifica quindi che il pacchetto contenga effettivamente un header IPv4 prima di accedere ai relativi campi. Le due istruzioni seguenti estraggono gli indirizzi IP sorgente e destinazione dal pacchetto:

```
src_ip = ip_pkt.src
dst_ip = ip_pkt.dst
```

La condizione principale del firewall è:

```
if ((src_ip == IP_H1 and dst_ip == IP_H3) or
    (src_ip == IP_H3 and dst_ip == IP_H1)):
```

Essa verifica entrambe le direzioni della comunicazione. La prima parte controlla il traffico da h_1 verso h_3 , mentre la seconda parte controlla il traffico da h_3 verso h_1 . La policy risulta quindi bidirezionale: non è possibile avviare comunicazioni IP in nessuno dei due versi.

L'istruzione:

```
self.logger.warning("FIREWALL_DROP: %s->%s", src_ip,
                    dst_ip)
```

non modifica il comportamento della rete, ma scrive un messaggio nel log del controller. È utile durante la fase di test poiché permette di verificare che Ryu abbia riconosciuto e bloccato il traffico vietato.

La variabile `match` definisce i campi che devono essere confrontati dallo switch:

```
match = parser.OFPMatch(
    eth_type=0x0800,
    ipv4_src=src_ip,
    ipv4_dst=dst_ip
)
```

Il campo `eth_type=0x0800` identifica i pacchetti Ethernet che trasportano IPv4. I campi `ipv4_src` e `ipv4_dst` rendono la regola specifica per la coppia sorgente–destinazione che ha violato la policy. Di conseguenza, la regola non blocca gli altri host o gli altri servizi della rete.

La funzione:

```
self._add_flow(  
    datapath,  
    priority=300,  
    match=match,  
    actions=[]  
)
```

invia allo switch un messaggio OpenFlow `OFF_FLOW_MOD` e installa nella sua *Flow Table* una nuova regola. Il parametro `datapath` identifica lo switch sul quale la regola deve essere installata. Il parametro `priority=300` assegna alla policy firewall una priorità maggiore rispetto alle regole di policy routing (priorità 200), di MAC learning (priorità 1) e di *table-miss* (priorità 0).

Il parametro `actions=[]` rappresenta l'azione di *DROP*. In OpenFlow non è necessario utilizzare un'azione esplicita denominata *DROP*: se una regola viene trovata ma non contiene alcuna azione di inoltro, il pacchetto viene scartato.

Infine, l'istruzione:

```
return
```

interrompe l'esecuzione di `_packet_in_handler()` per il pacchetto corrente. Questo evita che il pacchetto, dopo l'installazione della regola firewall, venga elaborato dal MAC learning o inoltrato tramite un messaggio `PacketOut`. Il primo pacchetto viene quindi bloccato dal controller, mentre i pacchetti successivi vengono bloccati direttamente dalla flow rule installata nello switch.

4 TASK 3: Policy Routing con Rete ad Anello

L'obiettivo del Task 3 è implementare un *policy routing* asimmetrico in cui il controller Ryu, non si limita più a inoltrare i pacchetti in base al MAC learning, ma decide intenzionalmente quale percorso devono seguire in base agli indirizzi IP sorgente e destinazione.

- **Andata** `h1 → h2`: il traffico percorre il cammino `h1 → s1 → s3 → s2 → h2`
- **Ritorno** `h2 → h1`: il traffico percorre il cammino diretto `h2 → s2 → s1 → h1`
- La politica di **firewall** tra `h1` e `h3` già presente deve essere mantenuta

4.1 Architettura logica del controller

Il Task 3 richiede l'integrazione di più componenti nella stessa applicazione Ryu: le regole di firewall del Task 2, il policy routing IPv4, la gestione delle richieste ARP nell'anello e il MAC learning di fallback. La Figura 2 mostra l'ordine logico con cui il controller analizza un pacchetto ricevuto tramite un messaggio `PacketIn`.

Le sezioni successive descrivono in dettaglio i singoli blocchi dello schema. In particolare, le flow statiche gestiscono il traffico IPv4 tra h_1 e h_2 , mentre la logica ARP controlla la propagazione dei broadcast sui link inter-switch per evitare loop di livello 2.

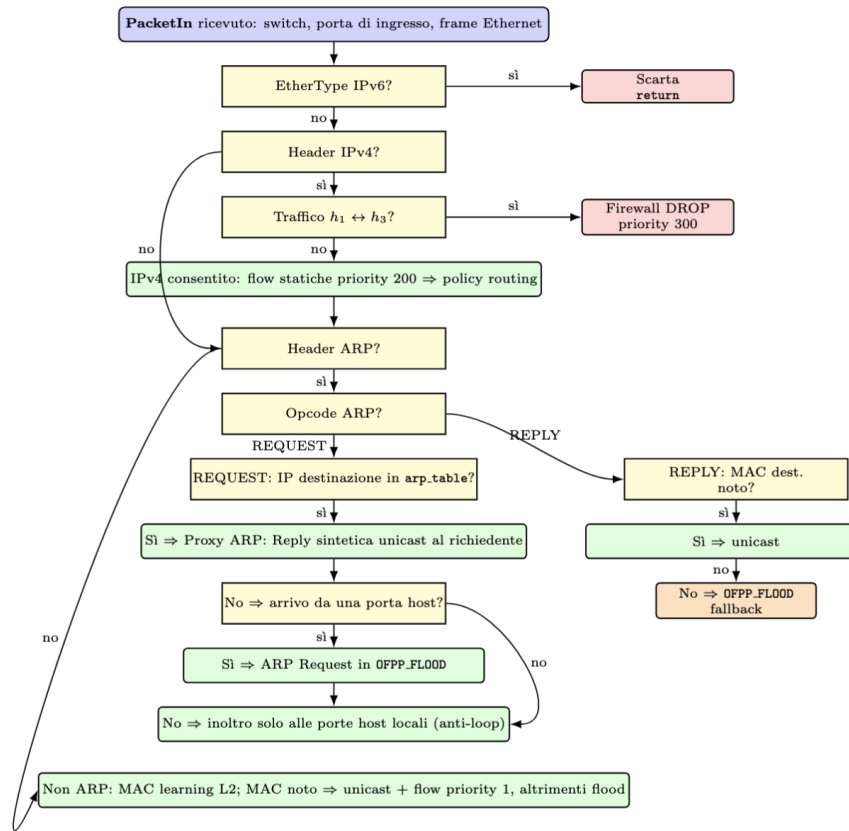


Figure 2: Schema a blocchi della logica del controller Ryu. Il diagramma riassume la gestione dei pacchetti IPv6, il controllo firewall tra h_1 e h_3 , il policy routing IPv4, la gestione delle ARP Request mediante flooding controllato e Proxy ARP, nonché l'inoltro delle ARP Reply e il MAC learning di fallback.

4.2 Fase 1: Identificazione di Host e Switch

La prima modifica consiste nella definizione delle costanti necessarie a identificare gli endpoint e gli switch coinvolti nella policy:

Listing 6: Costanti per il Task 3

```
IP_H1 = "10.0.0.10"
IP_H2 = "10.0.0.20"

DPID_S1 = 1
DPID_S2 = 2
DPID_S3 = 3
```

Le costanti `IP_H1` e `IP_H2` identificano i due host per i quali deve essere applicata la policy. Le costanti `DPID_S1`, `DPID_S2` e `DPID_S3` identificano invece gli switch OpenFlow. Il DPID viene ricevuto dal controller quando uno switch si connette e permette di installare ogni regola sul dispositivo corretto.

4.3 Fase 2: Descrizione della Policy di Routing

Il dizionario `STATIC_ROUTES` rappresenta la policy di routing. Ogni chiave è composta da una tripla:

(DPID switch, IP sorgente, IP destinazione)

Il valore associato è la porta OpenFlow di uscita da usare sullo switch indicato.

Listing 7: Policy di routing statico asimmetrico

```
STATIC_ROUTES = {
    # Andata: H1 -> S1 -> S3 -> S2 -> H2
    (DPID_S1, IP_H1, IP_H2): 4,
    (DPID_S3, IP_H1, IP_H2): 2,
    (DPID_S2, IP_H1, IP_H2): 2,

    # Ritorno: H2 -> S2 -> S1 -> H1
    (DPID_S2, IP_H2, IP_H1): 1,
    (DPID_S1, IP_H2, IP_H1): 2,
}
```

Le prime tre entry implementano il percorso di andata. In particolare, la prima regola invia su `s1` il traffico da `h1` a `h2` sulla porta 4, corrispondente al collegamento verso `s3`. Su `s3`, la porta 2 inoltra il pacchetto verso `s2`; infine, su `s2`, la porta 2 consegna il pacchetto a `h2`.

Le ultime due entry implementano il ritorno. Su `s2`, la porta 1 invia il traffico di `h2` direttamente verso `s1`; su `s1`, la porta 2 consegna il traffico a `h1`. Non viene definita alcuna regola su `s3` per la direzione $h_2 \rightarrow h_1$, poiché il ritorno non deve attraversare `s3`.

4.4 Fase 3: Installazione Proattiva delle Flow Rule

Le route non vengono installate dopo il primo ping, ma immediatamente quando uno switch stabilisce la connessione OpenFlow con il controller. Questa scelta riduce la latenza del primo pacchetto IP e garantisce che il percorso sia già disponibile prima dell'inizio della comunicazione.

Listing 8: Installazione della table-miss rule e delle route statiche

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures ,
            CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.
                                      OFPP_CONTROLLER,
                                      ofproto.
                                      OFPCML_NO_BUFFER)]

    self._add_flow(datapath, priority=0, match=match,
                  actions=actions)

    self._install_static_routes(datapath)
```

Il decoratore `@set_ev_cls(...EventOFPSwitchFeatures...)` indica che la funzione viene eseguita durante la fase di configurazione della connessione OpenFlow. La prima flow rule, con priorità 0, è la regola di *table-miss*: qualsiasi pacchetto non riconosciuto dalle altre regole viene inviato al controller. La chiamata a `_install_static_routes(datapath)` installa invece le regole di policy routing sullo switch che si è appena connesso.

Il metodo che effettua l'installazione è il seguente:

Listing 9: Metodo di installazione proattiva delle route

```
def _install_static_routes(self, datapath):
    parser = datapath.ofproto_parser
    dpid = datapath.id

    for (sw_dpid, src_ip, dst_ip), out_port in STATIC_ROUTES
        .items():
        if sw_dpid != dpid:
            continue

        match = parser.OFPMatch(
            eth_type=0x0800,
            ipv4_src=src_ip,
            ipv4_dst=dst_ip
        )
        actions = [parser.OFPACTIONOutput(out_port)]
        self._add_flow(datapath, priority=200,
                       match=match, actions=actions)
```

La variabile `dpid = datapath.id` identifica lo switch corrente. Il ciclo percorre tutte le entry di `STATIC_ROUTES`; la condizione `if sw_dpid != dpid` evita di installare su uno switch regole destinate a un altro dispositivo. Per ogni route pertinente viene costruito un match IPv4 tramite `eth_type=0x0800`, `ipv4_src` e `ipv4_dst`. L'azione `OFPACTIONOutput(out_port)` inoltra il pacchetto verso la porta definita dalla policy.

Le route hanno priorità 200. Tale valore è inferiore alla priorità 300 del firewall del Task 2, pertanto una policy di sicurezza ha sempre precedenza sul routing. La priorità 200 è però superiore alle regole di MAC learning (priorità 1) e alla table-miss rule (priorità 0), per cui il percorso imposto nel Task 3 prevale sul forwarding Ethernet generico.

4.5 Fase 4: Necessità della Gestione ARP

Le route del Task 3 fanno match esclusivamente su pacchetti IPv4 (`eth_type=0x0800`). Prima di inviare un pacchetto IP verso h_2 , h_1 deve tuttavia conoscere il suo indirizzo MAC. A questo scopo invia una richiesta ARP, che utilizza EtherType `0x0806` e non è quindi intercettata dalle route statiche.

Nella topologia lineare originale, il flooding ARP era sufficiente. Dopo l'aggiunta di `s3`, però, esiste l'anello `s1 -- s2 -- s3 -- s1`. Un flooding Ethernet standard può far circolare ripetutamente lo stesso frame broadcast nell'anello, creando un loop Layer 2 o una *broadcast storm*. Per questo è stata aggiunta una gestione ARP controllata dal controller.

4.6 Esempio di funzionamento di ARP sull'anello

Per comprendere gli effetti della topologia ad anello, si considera il caso in cui h_1 esegua un ping verso h_2 .

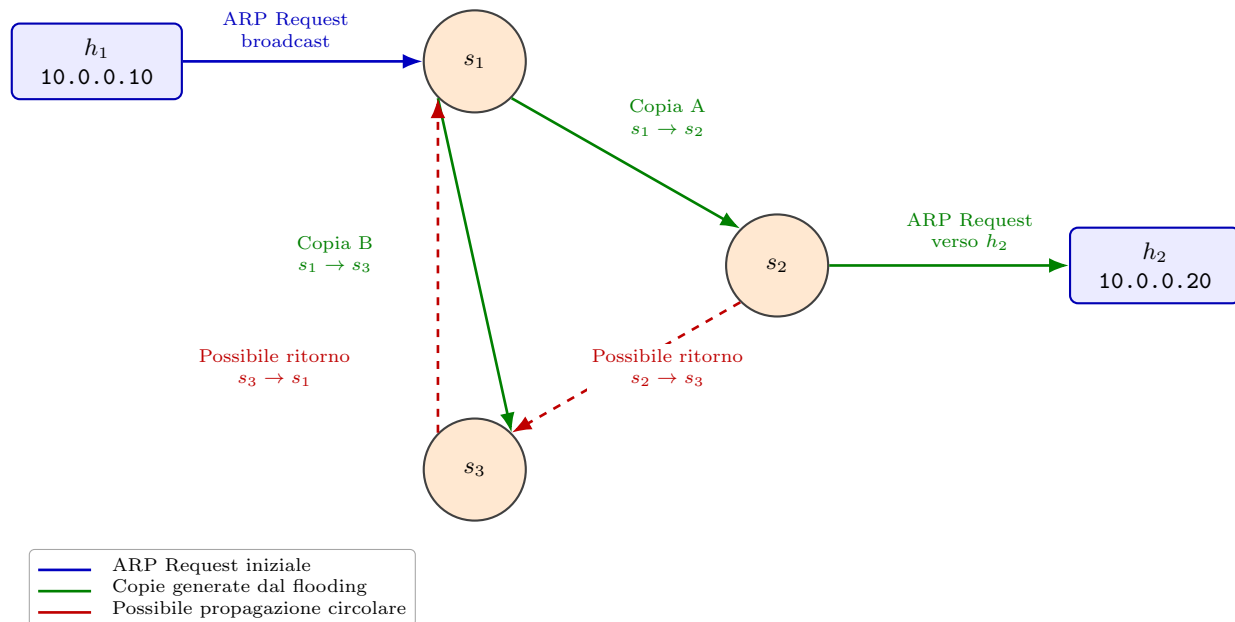


Figure 3: Esempio di flooding ARP nella topologia ad anello. La richiesta broadcast inviata da h_1 può essere duplicata da s_1 e propagata verso s_2 e s_3 . Le frecce rosse tratteggiate rappresentano la possibile circolazione del broadcast nell'anello in assenza di controllo.

1. h_1 invia una ARP Request broadcast a s_1 . Il frame ha come destinazione Ethernet `ff:ff:ff:ff:ff:ff`, quindi s_1 non può inoltrarlo verso una sola porta conosciuta.
2. s_1 genera un `PACKET_IN` verso Ryu, contenente la ARP Request.
3. Ryu inserisce l'associazione $h_1 \rightarrow s_1$, porta 1 nella propria tabella (`mac_to_port`).
4. Poiché s_1 ha due link verso gli altri switch dell'anello, un `PACKET_OUT` in modalità FLOOD non controllata produce due copie della stessa richiesta.
 - **Copia A:** inviata da s_1 verso s_2 ;
 - **Copia B:** inviata da s_1 verso s_3 .
5. La Copia A arriva a s_2 dalla porta collegata a s_1 . Se s_2 continuasse ad applicare il flooding verso *tutte* le proprie porte (compresa quella verso

s3), la richiesta proseguirebbe nell'anello invece di fermarsi dopo aver raggiunto h_2 .

6. Analogamente, la Copia B arriva a s3 dalla porta collegata a s1. Se s3 floodasse su tutte le porte, invierebbe una terza copia verso s2.
7. A questo punto s2 può ricevere due volte la stessa ARP Request relativa a h_1 : una prima volta tramite s1 (Copia A) e una seconda volta tramite s3 (Copia B rimbalzata). Ogni volta il campo Ethernet sorgente del frame è identico: il MAC di h_1 .
8. Ogni copia ricevuta da un nuovo switch genera un nuovo `PACKET_IN` verso Ryu. Il controller, eseguendo la stessa istruzione di apprendimento usata per il traffico legittimo,

```
self.mac_to_port[dpid][src_mac] = in_port
```

aggiorna la tabella ogni volta che riceve un nuovo `PACKET_IN`, indipendentemente dal fatto che si tratti di una copia duplicata dello stesso broadcast originario.

9. Poiché un frame Ethernet non possiede un campo TTL analogo a quello dei pacchetti IP, la stessa richiesta continua a circolare nell'anello, rientrando più volte negli stessi switch da porte diverse.

4.6.1 Il problema centrale: sovrascrittura delle porte e corruzione della memoria di Ryu

La tabella `mac_to_port` di Ryu associa a ciascun MAC *un'unica porta per switch*, cioè registra soltanto l'ultima porta da cui quel MAC è stato osservato:

```
mac_to_port[dpid][MAC_h1] = ultima_porta_osservata
```

Supponiamo che s3 veda per la prima volta il MAC di h_1 arrivare dalla porta collegata a s1 (Copia B, passo diretto). Ryu registra:

```
mac_to_port[s3][MAC_h1] = porta_verso_s1
```

Se però, a causa del rientro nell'anello, lo stesso MAC di h_1 arriva successivamente a s3 anche dalla porta collegata a s2 (perché la copia A ha fatto il giro $s1 \rightarrow s2 \rightarrow s3$), Ryu sovrascrive l'associazione precedente:

```
mac_to_port[s3][MAC_h1] = porta_verso_s2
```

A questo punto la memoria del controller è *corrotta*: Ryu crede che h_1 si trovi dietro la porta che collega s3 a s2, mentre in realtà h_1 non è mai stato fisicamente spostato. Questo fenomeno è noto come **MAC flapping**: lo stesso indirizzo MAC sembra "saltare" da una porta all'altra, perché il controller osserva copie dello stesso frame broadcast arrivate da direzioni diverse dell'anello,

e non ha modo di distinguere quale delle due porte rappresenti la posizione reale dell'host.

Le conseguenze pratiche sono analoghe a quelle di un broadcast storm in una rete Ethernet legacy senza STP:

- eventuali risposte future dirette a h_1 potrebbero essere inoltrate dal controller sulla porta sbagliata, cioè verso l'altro switch dell'anello invece che verso l'host;
- il numero di `PACKET_IN` elaborati dal controller cresce inutilmente, aumentando il carico computazionale su Ryu;
- la tabella non converge verso uno stato stabile, ma continua a essere riscritta a ogni nuovo giro del broadcast nell'anello;
- nel caso limite, la stessa richiesta ARP può continuare a circolare indefinitamente tra `s1`, `s2` e `s3`, saturando la capacità dei link.

4.7 Fase 5: Identificazione delle Porte Inter-Switch

Nella soluzione proposta il controller Ryu controlla la propagazione delle richieste ARP distinguendo le porte collegate agli switch dalle porte collegate agli host. Questa informazione è memorizzata nel dizionario `SWITCH_PORTS`:

Listing 10: Porte inter-switch utilizzate per il controllo del flooding ARP

```
SWITCH_PORTS = {
    DPID_S1: {1, 3, 4},
    DPID_S2: {1, 4},
    DPID_S3: {1, 2},
}
```

Quando una ARP Request entra da una porta host, il controller la propaga verso la rete per permettere al destinatario di riceverla. Quando invece la richiesta arriva da una porta inter-switch, il controller non la reinvia verso gli altri switch: la inoltra soltanto sulle porte host locali. In questo modo la richiesta può raggiungere h_2 , ma non può continuare a rimbalzare lungo l'anello.

Inoltre, il controller memorizza nella tabella `arp_table` le associazioni IP-MAC apprese osservando i messaggi ARP, ad esempio `10.0.0.20 → MAC` di h_2 . Durante il primo tentativo di comunicazione tra h_1 e h_2 , tale associazione non è ancora disponibile. La ARP Request di h_1 deve quindi essere propagata con flooding controllato fino a raggiungere h_2 , che genera la normale ARP Reply unicast verso h_1 . Dopo questo scambio, sia h_1 sia il controller conoscono il MAC di h_2 . Il meccanismo di *Proxy ARP* non è necessario per i successivi pacchetti del ping, poiché h_1 usa il MAC già presente nella propria cache ARP. Interviene soltanto se viene generata una nuova ARP Request, ad esempio dopo la scadenza o la cancellazione della cache ARP di h_1 . Se l'IP richiesto è già presente in `arp_table`, Ryu agisce da intermediario: invece di propagare nuovamente il

broadcast nell'anello, costruisce una ARP Reply sintetica contenente il MAC già noto di h_2 e la invia unicast a h_1 . In questo modo l'anello fisico rimane disponibile per applicare i percorsi IPv4 definiti dal policy routing, mentre le richieste ARP successive non generano ulteriori copie broadcast sui link inter-switch. Il controllo del flooding per le richieste iniziali e il Proxy ARP per quelle successive riducono quindi il rischio di loop di livello 2 e di aggiornamenti incoerenti della tabella MAC del controller.

4.8 Fase 6: Apprendimento ARP e Proxy ARP

Nel costruttore del controller è stata aggiunta una tabella ARP globale:

Listing 11: Tabella ARP globale del controller

```
self.arp_table = {}
```

La tabella memorizza associazioni nella forma IP -> MAC.

Listing 12: Apprendimento delle informazioni ARP

```
arp_pkt = pkt.get_protocol(arp.arp)
if arp_pkt:
    src_mac = arp_pkt.src_mac
    src_ip = arp_pkt.src_ip
    dst_ip = arp_pkt.dst_ip

    self.arp_table[src_ip] = src_mac
    self.mac_to_port[dpid][src_mac] = in_port
```

Quando il controller riceve un pacchetto ARP, estrae gli indirizzi necessari e aggiorna sia la tabella ARP sia la tabella di MAC learning:

Listing 13: Proxy ARP quando il MAC è noto

```
if dst_ip in self.arp_table:
    reply_mac = self.arp_table[dst_ip]
    e = ethernet.ethernet(dst=src_mac, src=reply_mac,
                          ethertype=0x0806)
    a = arp.arp(opcode=arp.ARP_REPLY,
                src_mac=reply_mac, src_ip=dst_ip,
                dst_mac=src_mac, dst_ip=src_ip)
    p = packet.Packet()
    p.add_protocol(e)
    p.add_protocol(a)
    p.serialize()
    self._packet_out(datapath, ofproto.OFPP_CONTROLLER,
                    [parser.OFPAActionOutput(in_port)], p.
                    data)

    return
```

Se il MAC richiesto è già noto, il controller costruisce un ARP Reply sintetico e lo invia direttamente all'host richiedente. Questo è Il *Proxy ARP* permette

a Ryu di rispondere al posto dell'host richiesto. Se il controller conosce già l'associazione IP-MAC nella tabella `arp_table`, genera una ARP Reply sintetica e la invia unicast al richiedente, evitando di propagare nuovamente la ARP Request broadcast nell'anello.

4.9 Fase 7: ARP REQUEST CON FLOODING CONTROLLATO ANTI-LOOP

Se il controller non conosce ancora il MAC richiesto, deve permettere alla richiesta ARP di raggiungere l'host destinatario. La logica introdotta distingue la provenienza del pacchetto:

Listing 14: Flooding ARP controllato

```
sw_ports = SWITCH_PORTS.get(dpid, set())

if in_port not in sw_ports:
    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    self._packet_out(datapath, in_port, actions, msg.data,
                     msg.buffer_id)
else:
    host_ports = []
    for p_num in range(1, 5):
        if p_num not in sw_ports:
            host_ports.append(p_num)

    for hp in host_ports:
        if hp != in_port:
            self._packet_out(datapath, in_port,
                             [parser.OFPActionOutput(hp)],
                             msg.data)

return
```

Se la richiesta ARP entra da una porta host, il controller la propaga verso la rete tramite flooding, affinché possa raggiungere host eventualmente collegati a switch diversi. Se invece la richiesta entra da una porta inter-switch, significa che ha già attraversato almeno un link della rete; in questo caso il controller la invia soltanto alle porte host locali e non la reinoltra verso altri switch. In questo modo viene interrotta la possibile circolazione del broadcast lungo l'anello.

4.10 Fase 8: ARP Reply e MAC Learning generico

Quando h_2 riceve la ARP Request, genera una ARP Reply unicast verso h_1 . La risposta viene quindi ricevuta da uno switch e inviata a Ryu tramite un nuovo `PacketIn`. Il controller aggiorna la tabella `mac_to_port` con la porta da cui ha osservato il MAC sorgente e controlla se conosce la porta associata al MAC di h_1 . Se tale associazione è presente, Ryu inoltra la ARP Reply unicast verso la porta corrispondente. Se invece il MAC di destinazione non è ancora noto, il codice utilizza `OFPP_FLOOD` come fallback. Nella topologia ad anello questo caso deve essere considerato con attenzione, poiché il flooding della ARP Reply non applica il filtro sulle porte inter-switch e può generare copie del frame nell'anello.

Listing 15: Inoltro di una risposta ARP

```
elif arp_pkt.opcode == arp.ARP_REPLY:
    self.mac_to_port[dpid][src] = in_port
    if dst in self.mac_to_port[dpid]:
        out_port = self.mac_to_port[dpid][dst]
        self._packet_out(datapath, in_port,
                        [parser.OFPActionOutput(out_port)],
                        msg.data, msg.buffer_id)
    else:
        self._packet_out(datapath, in_port,
                        [parser.OFPActionOutput(ofproto.
                                                OFPP_FLOOD)],
                        msg.data, msg.buffer_id)
    return
```

MAC learning generico. Tale blocco è il comportamento di fallback per i frame Ethernet generici non coperti dalle policy precedenti:

Listing 16: MAC learning generico

```
self.mac_to_port[dpid][src] = in_port
out_port = (self.mac_to_port[dpid][dst]
            if dst in self.mac_to_port[dpid]
            else ofproto.OFPP_FLOOD)
actions = [parser.OFPActionOutput(out_port)]

if out_port != ofproto.OFPP_FLOOD:
    match = parser.OFPMatch(in_port=in_port,
                            eth_dst=dst,
                            eth_src=src)
    self._add_flow(datapath, priority=1,
                  match=match, actions=actions)
```

Il controller associa il MAC sorgente alla porta di ingresso e, se conosce il MAC di destinazione, inoltra il frame sulla porta appropriata. Quando il percorso è noto, installa una regola Layer 2 con priorità 1 per evitare ulteriori `PacketIn`. Questa priorità è inferiore a quella del firewall (300) e del policy routing (200), per cui il MAC learning non può modificare il percorso imposto dalla policy.

5 Validazione sperimentale e risultati

5.1 Ambiente di test e metodologia

I test sono stati eseguiti nella topologia Kathará composta dagli switch OpenFlow $s1$, $s2$ e $s3$, dal controller Ryu e dagli host $h1$, $h2$ e $h3$. In particolare, sono stati verificati il blocco selettivo tra $h1$ e $h3$, la raggiungibilità di $h2$, il percorso asimmetrico imposto al traffico IPv4 e il corretto comportamento della gestione ARP nell'anello.

Prima di ogni prova, la topologia è stata avviata e gli switch sono stati connessi al controller. Gli output dei comandi sono stati raccolti nei nodi interessati e confrontati con le Flow Table di Open vSwitch.

5.2 Avvio del controller e connessione degli switch

Come prima verifica è stato avviato il controller Ryu sulla porta OpenFlow TCP 6653. L'obiettivo del test è confermare il corretto caricamento dell'applicazione, la connessione dei tre switch OpenFlow e l'installazione proattiva delle route IPv4 definite nel dizionario `STATIC_ROUTES`.

```
root@controller:/lab# ryu-manager --ofp-tcp-listen-port 6653 /ryu_campus.py
loading app /ryu_campus.py
loading app ryu.controller.ofp_handler
instantiating app /ryu_campus.py of RyuCampusController
instantiating app ryu.controller.ofp_handler of OFPHandler
[dpid=0000000000000003] ROUTE: 10.0.0.10→10.0.0.20 porta 2
>>> Switch connesso: dpid=0000000000000003
[dpid=0000000000000001] ROUTE: 10.0.0.10→10.0.0.20 porta 4
[dpid=0000000000000001] ROUTE: 10.0.0.20→10.0.0.10 porta 2
>>> Switch connesso: dpid=0000000000000001
[dpid=0000000000000002] ROUTE: 10.0.0.10→10.0.0.20 porta 2
[dpid=0000000000000002] ROUTE: 10.0.0.20→10.0.0.10 porta 1
>>> Switch connesso: dpid=0000000000000002
```

Figure 4: Output del controller Ryu all'avvio. I messaggi confermano la connessione degli switch $s1$, $s2$ e $s3$ e l'installazione proattiva delle route statiche di policy routing.

L'output conferma che i tre switch sono riconosciuti dal controller: $s1$ ha DPID 1, $s2$ ha DPID 2 e il nuovo switch $s3$ ha DPID 3. Per ciascun dispositivo, il messaggio `ROUTE` viene stampato durante l'esecuzione del metodo `_install_static_routes()`. Di conseguenza, le flow rule del policy routing sono installate al momento della connessione OpenFlow, prima della generazione dei pacchetti IP.

In particolare, il traffico $h1 \rightarrow h2$ viene configurato su $s1$ verso la porta 4, su $s3$ verso la porta 2 e su $s2$ verso la porta 2. Il traffico di ritorno $h2 \rightarrow h1$ viene invece configurato su $s2$ verso la porta 1 e su $s1$ verso la porta 2. L'assenza di una route di ritorno su $s3$ conferma il comportamento asimmetrico previsto.

5.3 Verifica dello stato iniziale delle Flow Table

5.4 Verifica iniziale delle Flow Table

Subito dopo la connessione degli switch al controller, prima di generare qualsiasi traffico di test, sono state ispezionate le Flow Table di *s1*, *s2* e *s3* tramite il comando:

```
ovs-ofctl -O OpenFlow13 dump-flows sX
```

```
End Ovsctl Command Log
[root@s1:~# ovs-ofctl -O OpenFlow13 dump-flows s1 ]
cookie=0x0, duration=466.253s, table=0, n_packets=0, n_bytes=0, priority=200,ip
,nw_src=10.0.0.10,nw_dst=10.0.0.20 actions=output:eth4
cookie=0x0, duration=466.254s, table=0, n_packets=0, n_bytes=0, priority=200,ip
,nw_src=10.0.0.20,nw_dst=10.0.0.10 actions=output:eth1
cookie=0x0, duration=466.257s, table=0, n_packets=0, n_bytes=0, priority=0 acti
ons=CONTROLLER:65535
root@s1:~# █
```

Figure 5: Flow Table di *s1* subito dopo la connessione al controller, prima della generazione di traffico di test.

```
[root@s2:~# ovs-ofctl -O OpenFlow13 dump-flows s2 ]
cookie=0x0, duration=470.771s, table=0, n_packets=0, n_bytes=0, priority=200,ip
,nw_src=10.0.0.10,nw_dst=10.0.0.20 actions=output:eth1
cookie=0x0, duration=470.771s, table=0, n_packets=0, n_bytes=0, priority=200,ip
,nw_src=10.0.0.20,nw_dst=10.0.0.10 actions=output:eth0
cookie=0x0, duration=470.771s, table=0, n_packets=0, n_bytes=0, priority=0 acti
ons=CONTROLLER:65535
root@s2:~# █
```

Figure 6: Flow Table di *s2* subito dopo la connessione al controller, prima della generazione di traffico di test.

```
[root@s3:~# ovs-ofctl -O OpenFlow13 dump-flows s3 ]
cookie=0x0, duration=475.740s, table=0, n_packets=0, n_bytes=0, priority=200,ip
,nw_src=10.0.0.10,nw_dst=10.0.0.20 actions=output:eth1
cookie=0x0, duration=475.741s, table=0, n_packets=0, n_bytes=0, priority=0 acti
ons=CONTROLLER:65535
root@s3:~# █
```

Figure 7: Flow Table di *s3* subito dopo la connessione al controller, prima della generazione di traffico di test.

L'output conferma che le route di policy routing sono già presenti al momento della connessione OpenFlow, senza che sia stato inviato alcun pacchetto IP dagli host. Su ciascuno switch sono presenti soltanto due tipi di regola: la flow di policy routing con priorità 200 e la table-miss rule con priorità 0.

In particolare, su $s1$ sono installate entrambe le direzioni previste dal Task 3:

- $h_1 \rightarrow h_2$: `actions=output:eth4`, verso $s3$;
- $h_2 \rightarrow h_1$: `actions=output:eth1`, verso h_1 .

Su $s2$ sono presenti le regole simmetriche rispetto al collegamento con h_2 e con $s1$:

- $h_1 \rightarrow h_2$: `actions=output:eth1`, verso h_2 ;
- $h_2 \rightarrow h_1$: `actions=output:eth0`, verso $s1$.

Su $s3$ è presente soltanto la regola di andata:

- $h_1 \rightarrow h_2$: `actions=output:eth1`, verso $s2$.

L'assenza di una regola IPv4 per la direzione $h_2 \rightarrow h_1$ su $s3$ conferma che il ritorno non è previsto attraversare questo switch, in accordo con l'asimmetria della polic.

Tutti i contatori `n_packets=0` e `n_bytes=0` indicano che, in questo momento, nessuna delle regole ha ancora ricevuto traffico corrispondente: la sola presenza delle flow dimostra l'installazione proattiva, indipendente dal traffico generato dagli host.

5.5 Test di connettività e verifica del firewall

Dopo aver verificato l'installazione iniziale delle route statiche, sono stati eseguiti i seguenti test di connettività. L'obiettivo è verificare che il traffico tra h_1 e h_2 sia consentito, che la comunicazione tra h_1 e h_3 sia bloccata in entrambe le direzioni e che h_2 rimanga raggiungibile.

Listing 17: Comandi eseguiti per i test di connettività e firewall

```
# Terminale di h1: connettivita consentita verso h2
ping -c 4 10.0.0.20

# Terminale di h1: traffico vietato verso h3
ping -c 4 -W 1 10.0.0.30

# Terminale di h3: verifica del blocco nella direzione
    opposta
ping -c 4 -W 1 10.0.0.10

# Terminale di h2: h2 deve rimanere raggiungibile da
    entrambi gli host
ping -c 4 10.0.0.10
ping -c 4 10.0.0.30

# Terminali degli switch: verifica delle flow dopo i test
```



```
ovs-ofctl -O OpenFlow13 dump-flows s1
ovs-ofctl -O OpenFlow13 dump-flows s2
ovs-ofctl -O OpenFlow13 dump-flows s3
```

```
root@h1:/# ping -c 4 10.0.0.20
PING 10.0.0.20 (10.0.0.20) 56(84) bytes of data.
64 bytes from 10.0.0.20: icmp_seq=1 ttl=64 time=89.0 ms
64 bytes from 10.0.0.20: icmp_seq=2 ttl=64 time=1.76 ms
64 bytes from 10.0.0.20: icmp_seq=3 ttl=64 time=2.71 ms
64 bytes from 10.0.0.20: icmp_seq=4 ttl=64 time=2.68 ms

--- 10.0.0.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3020ms
rtt min/avg/max/mdev = 1.763/24.031/88.976/37.497 ms
```

Figure 8: Test di connettività consentita: h_1 raggiunge h_2 .

```
root@h2:/# ping -c 4 10.0.0.10
PING 10.0.0.10 (10.0.0.10) 56(84) bytes of data.
64 bytes from 10.0.0.10: icmp_seq=1 ttl=64 time=6.62 ms
64 bytes from 10.0.0.10: icmp_seq=2 ttl=64 time=2.31 ms
64 bytes from 10.0.0.10: icmp_seq=3 ttl=64 time=2.31 ms
64 bytes from 10.0.0.10: icmp_seq=4 ttl=64 time=2.54 ms

--- 10.0.0.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3018ms
rtt min/avg/max/mdev = 2.307/3.443/6.616/1.834 ms
root@h2:/# ping -c 4 10.0.0.30
PING 10.0.0.30 (10.0.0.30) 56(84) bytes of data.
64 bytes from 10.0.0.30: icmp_seq=1 ttl=64 time=11.9 ms
64 bytes from 10.0.0.30: icmp_seq=2 ttl=64 time=2.37 ms
64 bytes from 10.0.0.30: icmp_seq=3 ttl=64 time=1.72 ms
64 bytes from 10.0.0.30: icmp_seq=4 ttl=64 time=1.20 ms
```

Figure 9: Test di connettività consentita: h_2 comunica correttamente con h_1 e h_3 .

```

root@h1:/# ping -c 4 -W 1 10.0.0.30
PING 10.0.0.30 (10.0.0.30) 56(84) bytes of data.

--- 10.0.0.30 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3078ms

root@h3:/# ping -c 4 -W 1 10.0.0.10
PING 10.0.0.10 (10.0.0.10) 56(84) bytes of data.

--- 10.0.0.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3057ms

FIREWALL DROP: 10.0.0.10 <-> 10.0.0.30
[dpid=0000000000000001] ARP from 10.0.0.30(02:ff:8f:13:60:41) porta 4
FIREWALL DROP: 10.0.0.30 <-> 10.0.0.10

```

Figure 10: Test del firewall bidirezionale: il traffico ICMP tra h_1 e h_3 viene bloccato in entrambe le direzioni.

```

root@s1:/# ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=10867.766s, table=0, n_packets=8, n_bytes=784, priority=200,ip,nw_src=10.0.0.10,
,nw_dst=10.0.0.20 actions=output:eth4
cookie=0x0, duration=10867.767s, table=0, n_packets=8, n_bytes=784, priority=200,ip,nw_src=10.0.0.20
,nw_dst=10.0.0.10 actions=output:eth1
cookie=0x0, duration=9676.030s, table=0, n_packets=3, n_bytes=294, priority=300,ip,nw_src=10.0.0.10,
nw_dst=10.0.0.30 actions=drop
cookie=0x0, duration=10867.770s, table=0, n_packets=7, n_bytes=458, priority=0 actions=CONTROLLER:65
535
root@s1:/# █

root@s2:/# ovs-ofctl -O OpenFlow13 dump-flows s2
cookie=0x0, duration=10989.420s, table=0, n_packets=8, n_bytes=784, priority=200,ip,nw_src=10.0.0.10
,nw_dst=10.0.0.20 actions=output:eth1
cookie=0x0, duration=10989.420s, table=0, n_packets=8, n_bytes=784, priority=200,ip,nw_src=10.0.0.20
,nw_dst=10.0.0.10 actions=output:eth0
cookie=0x0, duration=9783.024s, table=0, n_packets=3, n_bytes=294, priority=300,ip,nw_src=10.0.0.30,
nw_dst=10.0.0.10 actions=drop
cookie=0x0, duration=9746.643s, table=0, n_packets=3, n_bytes=294, priority=1,in_port=eth1,d1_src=ae
:7e:87:5a:26:58,d1_dst=02:ff:8f:13:60:41 actions=output:eth2
cookie=0x0, duration=9746.639s, table=0, n_packets=3, n_bytes=294, priority=1,in_port=eth2,d1_src=02
:ff:8f:13:60:41,d1_dst=ae:7e:87:5a:26:58 actions=output:eth1
cookie=0x0, duration=10989.420s, table=0, n_packets=12, n_bytes=834, priority=0 actions=CONTROLLER:6
5535

root@s3:/# ovs-ofctl -O OpenFlow13 dump-flows s3
cookie=0x0, duration=11160.926s, table=0, n_packets=8, n_bytes=784, priority=200,ip,nw_src=10.0.0
.10,nw_dst=10.0.0.20 actions=output:eth1
cookie=0x0, duration=11160.927s, table=0, n_packets=2, n_bytes=120, priority=0 actions=CONTROLLER
:65535

```

Figure 11: Flow rule reattiva di firewall con priorità 300 e azione drop, installate dopo i tentativi di comunicazione tra h_1 e h_3 . Nel percorso considerato, il drop avviene già in s_1 , perciò su s_3 non compaiono flow specifiche per il flusso $h_1 \leftrightarrow h_3$.

I risultati confermano che la policy di firewall è selettiva e bidirezionale: la comunicazione tra h_1 e h_3 viene bloccata, mentre h_2 resta raggiungibile dagli altri host. Le Flow Table mostrano inoltre che, dopo il primo tentativo di comunicazione vietata, il controller installa flow rule con priorità 300 e azione **drop**. Tali regole prevalgono sulle route statiche di policy routing, che hanno priorità 200.

5.6 Verifica del policy routing asimmetrico

Il policy routing asimmetrico può essere verificato analizzando le Flow Table degli switch riportate nella sezione precedente. In particolare, è sufficiente individuare le regole IPv4 con priorità 200 e leggere i campi `nw_src`, `nw_dst` e l'azione `output`.

Per il traffico in andata, le regole con `nw_src=10.0.0.10` e `nw_dst=10.0.0.20` mostrano che i pacchetti provenienti da h_1 vengono inoltrati da s_1 verso s_3 , da s_3 verso s_2 e infine da s_2 verso h_2 . Il percorso risultante è:

$$h_1 \rightarrow s_1 \rightarrow s_3 \rightarrow s_2 \rightarrow h_2$$

Al contrario, per il traffico di ritorno, le regole con `nw_src=10.0.0.20` e `nw_dst=10.0.0.10` indicano che i pacchetti vengono inoltrati da s_2 direttamente verso s_1 e quindi verso h_1 . Il percorso inverso risulta quindi:

$$h_2 \rightarrow s_2 \rightarrow s_1 \rightarrow h_1$$

L'assenza su s_3 di una flow con sorgente 10.0.0.20 e destinazione 10.0.0.10 conferma che il traffico di ritorno non attraversa tale switch. .

5.7 Test dei servizi TCP: curl e iperf3

```

root@ext1:~# iperf3 -c 10.0.0.20
Connecting to host 10.0.0.20, port 5201
[ 5] local 192.168.1.100 port 38542 connected to 10.0.0.20 port 5201
[ ID] Interval      Transfer    Bitrate      Retr  Cwnd
[ 5]  0.00-1.00  sec  5.88 MBytes  49.3 Mbits/sec    0   294 KBytes
[ 5]  1.00-2.00  sec  5.65 MBytes  47.4 Mbits/sec    0   563 KBytes
[ 5]  2.00-3.00  sec  5.78 MBytes  48.5 Mbits/sec    0   833 KBytes
[ 5]  3.00-4.00  sec  5.97 MBytes  50.0 Mbits/sec    0   1.08 MBytes
[ 5]  4.00-5.00  sec  6.09 MBytes  51.1 Mbits/sec    0   1.34 MBytes
[ 5]  5.00-6.00  sec  5.00 MBytes  41.9 Mbits/sec    0   1.60 MBytes
[ 5]  6.00-7.00  sec  6.25 MBytes  52.4 Mbits/sec   52   827 KBytes
^C[ 5]  7.00-7.70  sec  3.75 MBytes  44.8 Mbits/sec    0   872 KBytes
-----
[ ID] Interval      Transfer    Bitrate      Retr
[ 5]  0.00-7.70  sec  44.4 MBytes  48.3 Mbits/sec   52
[ 5]  0.00-7.70  sec  0.00 Bytes  0.00 bits/sec
iperf3: interrupt - the client has terminated
root@ext1:~# curl http://10.0.0.20:8080
<h1>Welcome to h2 - ASDN Web Server</h1>

```

Figure 12: Test dei servizi TCP da $ext1$: trasferimento dati con `iperf3` e accesso HTTP al web server h_2 tramite `curl`.

Il comando `curl` restituisce correttamente la pagina HTML del web server h_2 , confermando che il servizio HTTP sulla porta 8080 è raggiungibile dal client esterno $ext1$. Il comando `iperf3` stabilisce inoltre una connessione TCP con il server sulla porta 5201 e trasferisce dati a una velocità media di circa 48.3 Mbit/s durante l'intervallo misurato. Il test è stato interrotto manualmente dopo circa sette secondi, come indicato dal messaggio `iperf3: interrupt`, senza compromettere la validità della verifica. I risultati dimostrano quindi la corretta connettività dei servizi TCP tra la rete esterna e il web server interno.

5.8 Verifica della gestione ARP e PROXY ARP

```
[dpid=0000000000000001] ARP from 10.0.0.10(ce:ea:70:f1:3f:49) porta 2
[dpid=0000000000000003] ARP from 10.0.0.10(ce:ea:70:f1:3f:49) porta 1
[dpid=0000000000000002] ARP from 10.0.0.10(ce:ea:70:f1:3f:49) porta 1
[dpid=0000000000000002] ARP from 10.0.0.20(2a:d3:8a:b2:e7:06) porta 2
[dpid=0000000000000001] ARP from 10.0.0.20(2a:d3:8a:b2:e7:06) porta 3
[dpid=0000000000000002] ARP from 10.0.0.20(2a:d3:8a:b2:e7:06) porta 2
[dpid=0000000000000002] ARP PROXY REPLY: 10.0.0.10 is ce:ea:70:f1:3f:49
```

Figure 13: Log del controller Ryu durante il primo ping da h_1 a h_2 dopo il riavvio della topologia.

Dopo il riavvio dei container, sia le cache ARP degli host sia la tabella ARP del controller Ryu sono inizialmente vuote. Quando h_1 esegue il primo ping verso h_2 , non può ancora inviare direttamente il pacchetto IPv4: prima deve conoscere l'indirizzo MAC associato all'IP 10.0.0.20. Per questo h_1 genera una richiesta ARP broadcast.

I log visualizzati nella console di Ryu descrivono i frame ARP che arrivano al controller tramite messaggi OpenFlow `PacketIn`. Una riga nella forma

```
[dpid=...] ARP from IP(MAC) porta N
```

non rappresenta una risposta inviata dal controller. Essa indica invece che uno switch ha ricevuto un frame ARP sulla propria porta OpenFlow N e lo ha inviato a Ryu affinché venga elaborato. Il campo `dpid` identifica lo switch che ha generato il messaggio `PacketIn`; il campo `IP` identifica il mittente della richiesta o della risposta ARP osservata; il MAC tra parentesi è il MAC sorgente del frame ricevuto.

Ad esempio, il messaggio

```
[dpid=0000000000000001] ARP from
10.0.0.10(ce:ae:70:f1:3f:49) porta 2
```

indica che lo switch s_1 , identificato dal DPID 1, ha ricevuto un frame ARP in ingresso dalla porta 2. Il mittente del frame è h_1 , con indirizzo IP 10.0.0.10 e MAC `ce:ae:70:f1:3f:49`. Dopo aver ricevuto il `PacketIn`, il controller apprende e memorizza l'associazione:

```
10.0.0.10 -> ce:ae:70:f1:3f:49
```

Analogamente, la riga

```
[dpid=0000000000000001] ARP from  
10.0.0.20(2a:d3:8a:b2:e7:06) porta 3
```

indica che Ryu ha osservato un frame ARP il cui mittente è h_2 . Il controller apprende quindi anche l'associazione:

```
10.0.0.20 -> 2a:d3:8a:b2:e7:06
```

Il fatto che il frame di h_2 sia osservato su s_1 dalla porta 3 non significa che h_2 sia collegato fisicamente a s_1 . Significa che il frame ARP proveniente da h_2 ha attraversato s_2 ed è arrivato a s_1 dal link inter-switch associato alla porta 3.

La riga finale ha invece un significato differente:

```
[dpid=0000000000000002] ARP PROXY REPLY:  
10.0.0.10 is ce:ae:70:f1:3f:49
```

Questa riga non descrive un frame ricevuto, ma una risposta generata dal controller. Ryu ha ricevuto una richiesta ARP, tramite s_2 , per conoscere il MAC associato all'IP 10.0.0.10. Poiché l'associazione 10.0.0.10 -> ce:ae:70:f1:3f:49 era già stata appresa in precedenza, il controller non inoltra nuovamente la richiesta in broadcast. Esso crea direttamente una ARP Reply sintetica contenente l'informazione richiesta e la invia allo switch s_2 mediante un messaggio OpenFlow **PacketOut**. Lo switch inoltra infine la risposta in unicast verso l'host che aveva effettuato la richiesta.

La distinzione tra i due tipi di log è quindi la seguente:

- **ARP from:** frame ARP ricevuto dal controller tramite **PacketIn**; Ryu apprende l'associazione IP-MAC del mittente.
- **ARP PROXY REPLY:** frame ARP costruito dal controller e inviato tramite **PacketOut**; Ryu comunica un'associazione IP-MAC già presente nella propria tabella ARP.

Il vantaggio del Proxy ARP non consiste nel rendere unicast la risposta ARP, poiché anche una ARP Reply ordinaria generata dall'host destinatario è normalmente unicast. Il meccanismo evita invece la propagazione della ARP Request broadcast fino all'host richiesto e l'attesa della relativa risposta. Quando il MAC è già noto al controller, Ryu genera immediatamente una ARP Reply sintetica verso il richiedente, riducendo la latenza di risoluzione e il traffico broadcast nell'anello SDN.